

Gerenciamento de Recursos pelo Sistema Operacional

Um roteiro prático com comandos BASH para entender como o sistema operacional administra a CPU, a memória RAM e o armazenamento.

1. O Que É o Gerenciamento de Recursos

O sistema operacional (SO) é o software responsável por controlar os recursos de hardware de uma máquina. Dentre esses recursos, os três mais importantes para o seu funcionamento são a **CPU** (o processador, que executa instruções), a **memória RAM** (onde os programas em execução ficam carregados) e o **armazenamento** (o disco, onde dados e programas são guardados de forma permanente).

O papel principal do SO é **compartilhar** esses recursos finitos entre vários processos de forma *justa, eficiente e segura*. Se todos os programas pudessem usar a CPU e a memória livremente, haveria conflitos, corrupção de dados e travamentos. O kernel do SO esconde a complexidade do hardware e oferece uma interface padronizada para os programas — os chamados *system calls*.

Acesso aos recursos via BASH

Você não precisa programar o kernel. Pelo terminal BASH (shell), você consegue **observar** e **controlar** a forma como esses recursos estão sendo usados em tempo real. Todo o roteiro abaixo usa comandos simples que aparecem na maioria das distribuições Linux:

```
# Mostra a versão e informações do kernel
uname -a

# Lista os diretórios de informações do kernel
ls /proc/

# Informações gerais sobre o sistema em execução
hostnamectl
```

Dica

O diretório virtual `/proc` não existe de verdade no disco: ele é gerado na memória pelo kernel e expõe informações em tempo real sobre processos e recursos. Comandos como `top` e `free` leem essas informações internamente.

2. Gerenciamento da CPU CPU

A CPU executa apenas um conjunto de instruções por vez. Como quase sempre há mais processos prontos do que núcleos de processamento, o SO usa um **escalonador de processos** (scheduler) que decide, em cada instante, qual processo recebe o processador e por quanto tempo. Isso cria a ilusão de paralelismo (progresso simultâneo), mesmo em máquinas com poucos núcleos.

Observando o uso da CPU

O comando `top` mostra, em tempo real, os processos mais ativos e o percentual de CPU total. O `ps` gera uma fotografia estática dos processos. Já o `uptime` revela a *load average* (média de processos aguardando a CPU nos últimos 1, 5 e 15 minutos).

```
# Uso da CPU em tempo real (pressione 'q' para sair)
top
```

```
# Listar todos os processos com uso de CPU ordenado
ps aux --sort=-%cpu
```

```
# Ver o quanto o sistema está sobrecarregado
uptime
```

```
# Uso da CPU por núcleo, quando o pacote sysstat estiver instalado
mpstat -P ALL 1 3
```

EXEMPLO DE SAÍDA DO UPTIME

```
10:42:00 up 2 days,  4:10,  2 users,  load average: 0.31, 0.45, 0.62
```

A leitura `0.31 0.45 0.62` indica a média de processos na fila de pronto para execução. Crescer além do número de núcleos sugere que a CPU está saturada.

Controlando a prioridade dos processos

O escalonador leva em conta a **prioridade** de cada processo. O valor de *nice* varia de -20 (mais prioritário) até 19 (menos prioritário). Um processo com *nice* alto cede a CPU para outros processos mais importantes.

```
# Iniciar um processo com prioridade baixa (cede CPU)
nice -n 10 [[nome_do_processo]]
```

```
# Ajustar a prioridade de um processo já em execução
renice -n 15 -p [[id_do_processo]]
```

```
# Encerrar um processo que está consumindo muita CPU
kill -9 [[id_do_processo]]
```

```
# Procurar o PID de um processo pelo nome
pgrep my_program
```

Cuidado

`kill -9` (SIGKILL) encerra um processo sem permitir que ele salve dados ou libere recursos. Use antes `kill` simples (SIGTERM) para um encerramento limpo. Não execute `nice` com valores negativos em máquinas compartilhadas sem permissão.

Exercício 1

1. Execute `top` e identifique os três processos que mais usam CPU.
2. Inicie um comando `sleep 300 &`, depois use `ps` para achar o PID e `renice -n 5` nele.
3. Interprete a *load average* do seu sistema com `uptime`. O número está próximo de zero ou alto? O que isso indica?

3. Gerenciamento da Memória RAM RAM

A memória RAM guarda os dados e as instruções dos processos em execução. Como a RAM é menor que o espaço em disco, o SO combina RAM e uma área do disco chamada **swap** (ou *memória virtual*) para criar a impressão de haver mais memória do que fisicamente existe. Quando a RAM enche, o kernel move páginas de memória ociosas para a swap — processo chamado de *paginação*.

Observando a memória

```
# Resumo da memória física (RAM) e da swap
free -h

# Dados detalhados diretamente do kernel
cat /proc/meminfo

# Uso de memória e virtualização a cada segundo
vmstat 1 5

# Processos ordenados por consumo de memória
ps aux --sort=-%mem | head -n 10
```

EXEMPLO DE SAÍDA DO FREE -H

	total	used	free	shared	buff/cache	available
Mem:	7.7Gi	2.1Gi	1.1Gi	279Mi	4.5Gi	3.2Gi
Swap:	2.0Gi	0B	2.0Gi			2.0Gi

Observe a coluna **available**: ela considera que parte do *buff/cache* pode ser liberada quando necessário. Por isso "available" costuma ser maior que "free". A **swap usada** igual a zero indica que a RAM ainda não está esgotada.

Entendendo swap e paginação

```
# Se vmstat mostrar a coluna 'si' (swap in) e 'so' (swap out) com valores
# a máquina está com pouca memória e paginando com frequência (thrashing)
vmstat 1

# Abrir o 'top' e observar a memória de cada processo (colunas %MEM e RES)
top
```

Cuidado

Uso intenso de swap indica falta de RAM e degrada muito o desempenho, pois o disco é milhares de vezes mais lento que a RAM. Se a swap estiver sempre em uso, adicione RAM ou encerre processos desnecessários.

Exercício 2

1. Execute `free -h` e anote total, usada e disponível de RAM e swap.
2. Você acha que a swap está sendo usada? Como você concluiu isso?
3. Aplique `ps aux --sort=-%mem` e liste o processo que mais consome memória na sua sessão.

4. Gerenciamento do Armazenamento Disco

O armazenamento guarda dados de forma persistente em discos organizados em **partições**, cada uma com um **sistema de arquivos** (como ext4 ou xfs). Para ser acessado, um sistema de arquivos precisa ser **montado** (montagem, do inglês *mount*) em um diretório da árvore. Espaço, inodes e organização de pastas são aspectos essenciais desse gerenciamento.

Observando discos, partições e espaço

```
# Todos os discos e partições do sistema
lsblk

# Espaço usado e disponível nos sistemas de arquivos montados
df -h

# Identificação detalhada dos discos (requer permissão de root, use sudo
sudo fdisk -l

# Espaço usado por cada subdiretório de um caminho
du -h --max-depth=1 /home

# Espaço usado por um diretório específico
du -sh /home/

# Inodes: quantidade e uso (o número de arquivos e pastas)
df -i
```

EXEMPLO DE SAÍDA DO DF -H

Sist.Arq.	Tamanho	Usado	Disp.	Usa%	Montado em
/dev/sda1	238G	89G	139G	40%	/
/dev/sda2	512M	12M	501M	3%	/boot

Se a coluna **Uso%** alcançar valores acima de 90%, o disco pode ficar cheio a ponto de impedir novas gravações. Por isso profissionais monitoram tanto espaço quanto **inodes** (o limite de arquivos, que é separado do limite de bytes).

Um **inode (index node)** é a estrutura de dados que o sistema de arquivos do Linux/Unix usa para registrar as informações de um arquivo ou diretório. Você pode ter terabytes de espaço livre em disco, mas se o número de **inodes** esgotar (100% de uso), o sistema não deixará criar nenhum novo arquivo. Isso é muito comum em servidores que acumulam milhões de arquivos extremamente pequenos (como arquivos de sessão do PHP, registros de log temporários, pastas de e-mail ou caches). Nesses cenários, o disco não atinge o limite de gigabytes, mas o limite de inodes estoura.

Montagem e busca de arquivos grandes

```
# Ver quais sistemas de arquivos estão montados
mount | head -n 10

# Montar uma partição (exemplo didático, requer root)
sudo mount /dev/sdb1 /mnt

# Encontrar arquivos grandes que ocupam muito espaço
find /home -type f -size +100M

# Ver uso de disco de um arquivo ou pasta
stat arquivo_grande.iso
```

Cuidado

Montar ou desmontar partições (`mount` / `umount`) é uma operação privilegiada. Sempre desmonte antes de remover fisicamente um dispositivo e evite manipular partições do sistema fora de ambientes de teste.

Exercício 3

1. Execute `df -h` e `df -i`. O sistema está mais perto de esgotar bytes ou inodes?
2. Use `du -h --max-depth=1` no seu diretório pessoal e descubra qual subpasta ocupa mais espaço.
3. Com `find / -type f -size +500M` (use `sudo` se necessário), tente localizar arquivos grandes na máquina.

5. Projeto Final: Script de Monitoramento de Recursos

Para consolidar o aprendizado, você vai escrever um script BASH que **monitora os três tipos de recurso** (CPU, RAM e armazenamento) e gera um relatório formatado com data e hora. O script usa `top / ps` para CPU, `free` para RAM e `df` para armazenamento, salva a saída em um arquivo de log e repete a coleta a cada intervalo usando um loop.

Salve o conteúdo abaixo em um arquivo chamado `monitor_recursos.sh` :

```
#!/bin/bash
# monitor_recursos.sh - Monitora CPU, RAM e armazenamento e gera um log

LOG="registro_recursos.log"
INTERVALO=5    # segundos entre cada amostragem
AMOSTRAS=3     # número de coletas

echo "Iniciando monitoramento em $(date)" | tee "$LOG"

for ((i = 1; i <= AMOSTRAS; i++)); do
    echo "====="
    echo "Amostra $i em $(date '+%d/%m/%Y %H:%M:%S')"
    echo "-----"

    # 1) CPU: processo que mais está usando a CPU no momento
    echo "-- TOP 3 PROCESSOS POR CPU --"
    ps aux --sort=-%cpu | awk 'NR <= 4 {printf "%-8s %5s %s\n", $3"%", $2,

    # 2) RAM: total, usada e disponível
    echo "-- MEMÓRIA RAM --"
    free -h | awk '/^Mem:/ {print "Total:", $2, "| Usada:", $3, "| Livre:"
    echo "Memória em swap usada: $(free -m | awk '/Swap:/ {print $3}') MiB

    # 3) Armazenamento: uso da partição raiz /
    echo "-- ARMAZENAMENTO (partição /) --"
    df -h / | awk 'NR == 2 {print "Disponível:", $4, "| Uso:", $5}'

    echo "-----"
    echo "Registrado no arquivo: $LOG"
    sleep "$INTERVALO"

    if [ "$i" -lt "$AMOSTRAS" ]; then
        sleep "$INTERVALO"
    fi
done | tee -a "$LOG"
```

```
echo "Monitoramento concluído em $(date)" | tee -a "$LOG"
```

Como executar

```
# Tornar o script executável
chmod +x monitor_recursos.sh

# Executar o script
./monitor_recursos.sh

# Conferir o conteúdo do relatório gerado
cat registro_recursos.log
```

EXEMPLO PARCIAL DO RELATÓRIO GERADO

```
Iniciando monitoramento em sex 12 ago 10:45:00
=====
Amostra 1 em 12/08/2026 10:45:00
-----
-- TOP 3 PROCESSOS POR CPU --
0.3%   1234 firefox
0.2%   0987 bash
0.1%   1122 monitor_recursos
-- MEMÓRIA RAM --
Total: 7.7Gi | Usada: 2.1Gi | Livre: 1.1Gi
Memória em swap usada: 0 MiB
-- ARMAZENAMENTO (partição /) --
Disponível: 139G | Uso: 40%
-----
```

Dica

Para deixar o monitoramento contínuo, altere `AMOSTRAS` para um valor grande ou use `while true`. Também é possível agendar a execução com o `cron` do sistema, tema comum em administração de servidores.

Desafio extra

1. Modifique o script para avisar (imprimir um alerta) quando o uso do disco passar de 80%.
2. Use `top -b -n 1` dentro do script para capturar a carga de CPU da máquina inteira, não só de um processo.

3. Adicione a *load average* (`uptime`) ao relatório.

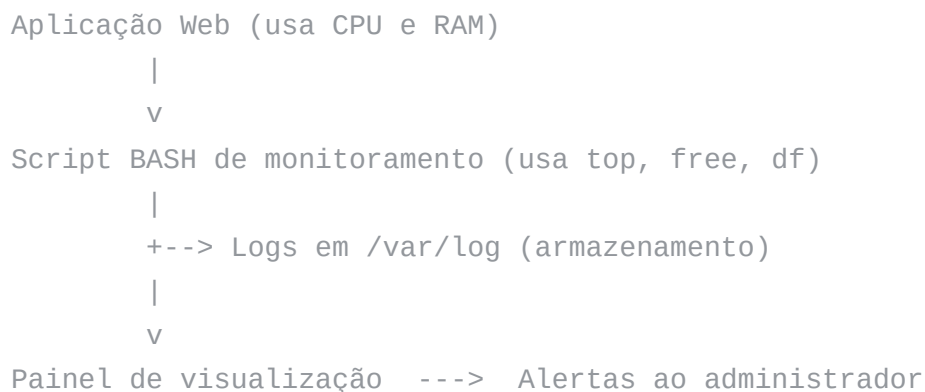
6. Casos de Uso Práticos

O gerenciamento e o monitoramento de recursos são atividades cotidianas em diversas áreas. Veja cenários reais:

- **Administração de servidores:** um administrador usa `top` , `free` e `df` para detectar quando um servidor web está sem memória ou com disco cheio, planejando upgrades.
- **Monitoramento (DevOps/SRE):** scripts como o do projeto coletam métricas periodicamente e alimentam ferramentas de alerta e painéis de observabilidade.
- **Execução de aplicações científicas:** rodar simulações pesadas exige controlar a prioridade (`nice` / `renice`) para não travar o sistema operacional de outros usuários.

Exemplo de arquitetura simples

Um servidor Linux com uma aplicação web é monitorado por um script que gera logs. Os logs alimentam um painel de visualização, e um sistema de alertas informa o administrador quando um limite é atingido.



7. Prós e Contras: Uma Análise de Trade-offs

Entender as vantagens e limitações ajuda a decidir quando (e quando não) usar cada abordagem de gerenciamento de recursos.

Aspecto	Prós	Contras / Limitações
Escalonamento de CPU	Distribui justa e eficientemente o processador entre processos; garante responsividade.	Processos com prioridade errada podem sofrer starvation (falta de CPU indefinida).
Memória virtual (swap)	Permite executar programas maiores que a RAM física e isolar processos.	<i>Thrashing</i> (paginação excessiva) degrada muito o desempenho; swap não substitui RAM física.
Particionamento / sistemas de arquivos	Isola dados, facilita backups e limita falhas a uma partição específica.	Particionamento rígido desperdiça espaço; preencher o disco pode impedir gravações inesperadamente.
Monitoramento via BASH	Simples, onipresente em Linux, fácil de automatizar em scripts e agendar via cron.	Logs podem crescer sem limite; comandos em tempo real exigem leitura manual frequente.

Quando NÃO usar cada recurso usual e a alternativa

- **Não use swap como solução definitiva** para falta de RAM: a alternativa adequada é adicionar memória física ou encerrar processos.
- **Não use `du` de forma recursiva no diretório raiz inteiro** sem filtros em máquinas grandes: a alternativa é limitar com `--max-depth` ou rodar fora do horário de pico.
- **Não use `nice -n -20` em prioridades negativas** sem permissão de administrador: a alternativa é respeitar os valores padrão e ajustar apenas o necessário com `renice`.
- **Não dependa apenas de `free`** para entender pressão de memória: a alternativa é cruzar com `vmstat` (colunas `si / so`) para detectar paginação real.